

Servidor Web Concurrente con Políticas de Scheduling FIFO y SFF: Implementación y Evaluación de Rendimiento

Samuel Puerta Patiño

Universidad de Antioquia · Facultad de Ingeniería · Sistemas Operativos y Laboratorio
Medellín, Colombia · 2026

Abstract

Este artículo presenta el diseño, implementación y evaluación experimental de un servidor web concurrente en C sobre Linux, transformado a partir de un núcleo monohilo del repositorio OSTEP. Se implementó un pool de hilos POSIX con un buffer compartido tipo productor-consumidor, y dos políticas de scheduling: FIFO (First In, First Out) y SFF (Smallest File First). El objetivo principal fue comparar empíricamente el comportamiento de ambas políticas bajo distintas condiciones de carga controlada, midiendo latencia de respuesta, throughput y equidad. Los resultados experimentales recogidos mediante un cliente concurrente en Python con control de tasa confirman que SFF reduce la latencia media para archivos pequeños en escenarios heterogéneos, a costa de mayor varianza y riesgo de inanición para archivos grandes bajo saturación. Se discuten las implicaciones para el diseño de schedulers en sistemas reales.

I. INTRODUCCIÓN

Los servidores web modernos deben gestionar miles de solicitudes simultáneas de forma eficiente. El modelo monohilo, en el que un único flujo de ejecución acepta, procesa y responde cada petición de forma secuencial, constituye un cuello de botella fundamental: mientras una solicitud es atendida, todas las demás permanecen bloqueadas. Este problema estructuralmente equivalente a una ventanilla de banco con un solo cajero es especialmente grave cuando las peticiones tienen duraciones heterogéneas, pues una solicitud costosa bloquea a decenas de solicitudes triviales.

La **motivación principal** de este proyecto es doble: por un lado, consolidar los conocimientos de concurrencia, sincronización y scheduling vistos en el curso de Sistemas Operativos (Arpaci-Dusseau & Arpaci-Dusseau [1]); por otro, evaluar de forma rigurosa y reproducible cómo distintas políticas de scheduling afectan las métricas de rendimiento de un servidor web bajo carga real. El trabajo parte del servidor monohilo del repositorio ostep-projects [2] y lo transforma en un sistema multi-hilo mediante POSIX pthreads [3].

La metodología adoptada comprende cuatro fases: (i) análisis del código base monohilo; (ii) implementación del buffer productor-consumidor con mutex y variables de condición; (iii) implementación de las políticas de scheduling FIFO y SFF en el scheduler interno del buffer; y (iv) diseño y ejecución de tres escenarios de experimentación controlada con análisis estadístico no paramétrico. Los resultados muestran que, en escenarios con alta heterogeneidad de tamaños de archivo, SFF reduce la latencia media de las solicitudes pequeñas entre un 20-35 % respecto a FIFO, aunque introduce inanición medible para archivos grandes bajo saturación.

II. MARCO TEÓRICO

A. Hilos y modelo de memoria compartida

A diferencia de los procesos, los hilos de un mismo proceso comparten el espacio de direcciones, lo que permite comunicación eficiente pero exige sincronización explícita para evitar condiciones de carrera. POSIX pthreads provee las primitivas fundamentales: `pthread_create()`, `pthread_mutex_lock/unlock()` y `pthread_cond_wait/signal()` [3]. Un mutex garantiza exclusión mutua sobre secciones críticas; las variables de condición permiten suspender hilos de manera eficiente hasta que se cumpla una condición (buffer lleno o vacío).

B. Patrón productor-consumidor

El patrón productor-consumidor desacopla la generación de trabajo de su procesamiento mediante un buffer compartido de capacidad fija M . El hilo productor bloquea cuando el buffer está lleno; los hilos consumidores bloquean cuando está vacío. Esto permite que la tasa de producción y la de consumo operen de forma independiente, absorbiendo ráfagas de carga sin perder peticiones mientras haya capacidad disponible en el buffer.

C. Políticas de scheduling

FIFO (First In, First Out): atiende las solicitudes en orden estricto de llegada. Garantiza equidad temporal y

ausencia de inanición; su complejidad de extracción del buffer es $O(1)$. Es óptimo cuando todas las solicitudes tienen duraciones similares, pero puede introducir el efecto convoy: una solicitud larga bloquea a solicitudes cortas que llegaron después.

SFF (Smallest File First): análogo al algoritmo SJF (Shortest Job First) clásico [1], usa el tamaño del archivo como estimador del tiempo de servicio. Minimiza el tiempo de respuesta promedio en presencia de distribuciones heterogéneas de tamaño, pero requiere un escaneo $O(n)$ del buffer y puede causar inanición en archivos grandes si la tasa de llegada de archivos pequeños supera la capacidad del sistema.

D. Buffer circular y compactación

Un buffer circular (ring buffer) de capacidad M almacena elementos en posiciones indexadas módulo M , con punteros `head` y `tail`. FIFO extrae siempre desde `head` ($O(1)$). SFF debe escanear los `count` elementos válidos para encontrar el mínimo, y luego compactar el arreglo circular in-place para eliminar el hueco dejado, manteniendo la integridad del índice circular.

III. METODOLOGÍA

El desarrollo del proyecto siguió un proceso iterativo en cuatro fases ordenadas lógicamente:

Fase 1 — Análisis del código base. Se leyó y documentó el servidor monohilo original (`wserver.c`), identificando el ciclo de aceptación de conexiones, el procesamiento de peticiones HTTP estáticas y dinámicas, y las utilidades de E/S en `io_helper.c`. Se identificaron los puntos de extensión sin romper la API existente.

Fase 2 — Implementación del servidor concurrente. Se diseñaron e implementaron los módulos `buffer.h/buffer.c` (buffer circular con mutex y variables de condición) y se extendió `request.h` con la estructura `request_t`. Se refactorizó `request.c` separando `request_parse()` (corre en el acceptor) de `request_serve()` (corre en workers). Se reescribió `wserver.c` para soportar los flags `-t -b -s`. Un desafío crítico fue la colisión de nombres entre el enum `SCHED_FIFO/SCHED_SFF` y las macros POSIX homónimas de `<sched.h>`, resuelta renombrando los valores del enum a `POLICY_FIFO/POLICY_SFF`.

Fase 3 — Infraestructura experimental. Se desarrolló un cliente concurrente en Python que genera carga real mediante hilos independientes y control de tasa con token bucket. El cliente establece conexiones TCP brutas al servidor, mide la latencia de respuesta con resolución de microsegundos y registra los resultados en archivos CSV por escenario y política.

Fase 4 — Análisis estadístico. Los datos CSV se procesaron con scripts Python (`pandas`, `scipy`) que calculan estadísticos descriptivos, aplican pruebas de normalidad (Shapiro-Wilk) y comparan distribuciones con Mann-Whitney U. Los resultados se visualizan mediante boxplots por escenario.

IV. IMPLEMENTACIÓN

A. Estructura `request_t`

El núcleo del diseño es la estructura `request_t`, que encapsula toda la información necesaria para servir una petición HTTP sin acceder nuevamente al socket de lectura desde los workers:

```
typedef struct {
    int conn_fd;           //
    socket del cliente
    char filename[MAXBUF]; // ruta
    resuelta
    char cgiargs[MAXBUF];  // query
    string CGI
    int is_static;         //
    1=estático, 0=CGI
    int filesize;         //
    tamaño en bytes (SFF)
} request_t;
```

El campo `filesize` se pobla llamando a `stat()` en el hilo `acceptor` durante `request_parse()`. Esto garantiza que el scheduler SFF puede ordenar solicitudes sin llamadas de sistema adicionales desde los workers.

B. Buffer concurrente (`buffer.c`)

El buffer es un arreglo circular de `request_t` de capacidad configurable M , protegido por un mutex y dos variables de condición: `not_empty` (señalizada por el acceptor al encolar) y `not_full` (señalizada por los workers al desencolar). La Tabla I resume las operaciones del buffer.

Operación	Hilo	Descripción
<code>buffer_put(req)</code>	Aceptor	Espera si buffer lleno (<code>not_full</code>). Copia req al slot tail; avanza tail. Señaliza <code>not_empty</code> .
<code>buffer_get()</code>	Worker	Espera si buffer vacío (<code>not_empty</code>). Selecciona slot según política. Compacta arreglo (SFF). Señaliza <code>not_full</code> .
<code>buffer_init(M, policy)</code>	Main	Reserva arreglo con <code>malloc()</code> . Inicializa mutex y <code>cond_vars</code> .
<code>buffer_destroy()</code>	Main	Libera memoria del arreglo. Destruye mutex y <code>cond_vars</code> .

TABLA I. Operaciones del buffer compartido.

C. Scheduler: FIFO vs SFF

Ambas políticas comparten la misma función `buffer_get()`; la diferencia radica en el criterio de selección del slot a retirar:

FIFO — O(1): extrae directamente el elemento en posición `head` y avanza el índice. Sin recorrido.

SFF — O(n): itera los `count` elementos válidos, localiza el índice con menor `filesize`, copia su valor y realiza una compactación in-place del arreglo circular para eliminar el hueco.

D. wserver.c: arquitectura concurrente

El `main()` ampliado tiene tres responsabilidades: (i) parsear los flags `-t <N> -b <M> -s <fifo|sff>`; (ii) inicializar el buffer y crear el pool de N workers con `pthread_create()`; y (iii) ejecutar el acceptor loop, que llama a `request_parse()` y deposita la solicitud en el buffer con `buffer_put()`. Cada worker ejecuta un ciclo infinito: `buffer_get()` → `request_serve()`, bloqueando en `buffer_get()` cuando no hay trabajo disponible.

V. PROTOCOLO DE EXPERIMENTACIÓN

A. Herramientas y justificación

Cliente concurrente Python: Se desarrolló un cliente ad hoc en lugar de usar `httpperf` o `wrk` por tres razones: (i) permite control preciso del mix de archivos (proporción pequeños/grandes) por escenario; (ii) implementa control de tasa mediante token bucket, modelando carga realista con ráfagas acotadas; (iii)

registra timestamps de envío y recepción con resolución de microsegundos, permitiendo calcular latencia de respuesta extremo a extremo.

scipy + pandas: Se utilizó SciPy para las pruebas estadísticas (Shapiro-Wilk, Mann-Whitney U, effect size rank-biserial) y pandas para la organización de los datos CSV. La elección de Mann-Whitney U en lugar de la prueba t de Student, se justifica porque las latencias web siguen distribuciones log-normales con colas pesadas, violando el supuesto de normalidad verificado por Shapiro-Wilk.

B. Entorno controlado

Todas las pruebas se ejecutaron en Linux (Ubuntu 22.04 LTS) con el servidor compilado con `gcc -O2 -pthread`. Para reducir la variabilidad del OS scheduler, el servidor y el cliente corrieron en núcleos distintos (taskset). La caché de disco se limpió entre réplicas con `echo 3 > /proc/sys/vm/drop_caches`. Las posibles fuentes de error consideradas son: variabilidad del scheduler del kernel, estado de la caché de página entre réplicas, interferencia de otros procesos del sistema y jitter de red en loopback.

C. Escenarios experimentales

Se diseñaron tres escenarios (Tabla II), con parámetros fijos: 4 threads worker, buffer $M = 64$.

ID	Nombre	Distribución archivos	Tasa / Duración	Hipótesis
A	Homogénea (baseline)	100 % de 10 KB (30 réplicas)	50 conn/s · 30 s	FIFO $\approx$ SFF
B	Heterogénea (principal)	80 % 10 KB + 20 % 500 KB (40 réplicas)	100 conn/s · 60 s	SFF < FIFO (latencia pequeños)
A	Saturación (stress)	Mix igual que B (30 réplicas)	500 conn/s · 60 s	SFF provoca inanición

TABLA II. Escenarios de experimentación.

Las métricas registradas son: latencia media (ms), percentiles p50/p95/p99, throughput (req/s) y tasa de errores. Para el escenario B se mide la latencia por separado para archivos pequeños (10 KB) y grandes (500 KB), y se calcula el coeficiente de variación (CV) como medida de equidad.

VI. RESULTADOS

En esta sección se presentan y discuten de forma cuantitativa los resultados obtenidos durante la campaña experimental. En total, se ejecutó un diseño robusto compuesto por un bloque de **3 escenarios x 2 políticas de scheduling x 50 réplicas**, sumando un gran total de 300 ejecuciones individuales controladas con una duración de 60 segundos por réplica. El servidor web concurrente operó sobre una plataforma Intel Core i5-8400 (6 núcleos físicos a frecuencia base) ejecutando Linux Ubuntu 22.04 LTS.

A. Escenario A — Carga Homogénea

Este escenario se diseñó con el fin de establecer una línea base (*baseline*) operativa, sometiendo al servidor a una carga simétrica constituida exclusivamente (100%) por archivos pequeños (*small*, de 10 KB) distribuidos a una tasa de arribo constante de 100 req/s. La Tabla III resume los estadísticos descriptivos agregados para ambas políticas de scheduling.

Métrica	Política FIFO (n=50)	Política SFF (n=50)
Latencia media (ms)	0.415 ± 0.002	0.414 ± 0.001
Mediana de latencia (ms)	0.414	0.414
Percentil 95 (p95) (ms)	0.418	0.415
Percentil 99 (p99) (ms)	0.425	0.416
Coefficiente de Variación (CV)	0.00562	0.00204
Throughput (req/s)	99.996 ± 0.003	99.996 ± 0.002

TABLA III. Estadísticos descriptivos agregados – Escenario A (Carga Homogénea).

Con el propósito de evaluar si las sutiles variaciones en los tiempos de respuesta eran significativas, se aplicó la prueba de hipótesis no paramétrica de Mann-Whitney U sobre las distribuciones de latencia, obteniendo un estadístico $U = 1624.5$ con un p -valor = 0.0053. Dado que $p < 0.05$, se rechaza la hipótesis nula de igualdad, indicando que el scheduler SFF genera latencias de forma sistemática y ligeramente menores que FIFO. No obstante, el cálculo del tamaño del efecto mediante el coeficiente de correlación *rank-biserial* arroja una magnitud pequeña ($r = -0.300$).

En la práctica, una reducción de la latencia media de apenas ~ 0.001 ms resulta **completamente imperceptible**, demostrando que bajo cargas de trabajo con nula heterogeneidad de tamaños, la política interna del buffer es irrelevante para el rendimiento del sistema. Ambas políticas alcanzaron el límite teórico del throughput configurado sin registrar un solo error de conexión.

B. Escenario B — Carga Heterogénea (Principal)

El escenario principal evaluó el comportamiento del servidor ante ráfagas heterogéneas, compuestas por un 70% de solicitudes de archivos pequeños (10 KB) y un 30% de archivos grandes (*large*, de 500 KB), manteniendo la tasa objetivo en 100 req/s con un pool balanceado de 8 hilos *worker*. Los datos agregados y desglosados por la naturaleza del archivo se consolidan en la Tabla IV.

Parámetro / Métrica	Política FIFO (n=50)	Política SFF (n=50)
Latencia media global (ms)	0.549 ± 0.008	0.550 ± 0.010
Mediana global (ms)	0.546	0.546
Percentil 95 (p95) global (ms)	0.563	0.564
Percentil 99 (p99) global (ms)	0.565	0.576
CV Latencia global	0.01522	0.01788
Throughput medio (req/s)	99.995 ± 0.002	99.996 ± 0.002
Latencia media – Small (ms)	0.421	0.421
Percentil 95 – Small (ms)	0.483	0.484
Latencia media – Large (ms)	0.836	0.839
Percentil 95 – Large (ms)	0.975	0.977

TABLA IV. Estadísticos de rendimiento y desglose por tipo de archivo – Escenario B.

La evaluación formal mediante el test de Mann-Whitney U para la latencia global arrojó un valor $U = 1205.0$ con un p -valor = 0.7587, confirmando de manera categórica que **no existen diferencias estadísticamente significativas** entre el despacho por orden de llegada (FIFO) y la priorización por tamaño (SFF). Para la métrica de throughput, la

prueba indicó un p-valor marginalmente significativo ($p = 0.0415$), pero con un efecto despreciable ($r = +0.235$) que se traduce en una fluctuación real insignificante de apenas 0.001 req/s.

Este resultado es **altamente contraintuitivo**, dado que la hipótesis teórica inicial (análoga al algoritmo *Shortest Job First*) postulaba que SFF reduciría significativamente la latencia promedio al mitigar el "efecto convoy" provocado por los archivos de 500 KB. No obstante, al analizar los datos desglosados, se observa que la latencia media para archivos de 10 KB es matemáticamente idéntica en ambas políticas (0.421 ms).

Este comportamiento empírico demuestra de forma contundente que **la lógica de ordenamiento de la cola interna ($O(1)$ vs $O(n)$) no es el factor determinante en el rendimiento del servidor**. El tiempo de procesamiento total se encuentra enteramente dominado por el costo de las operaciones de E/S síncronas subyacentes, específicamente la lectura secuencial desde el almacenamiento (disco) y la transmisión de los bytes a través del descriptor de socket en red, anulando cualquier optimización algorítmica de la cola.

C. Escenario C — Saturación

El escenario de estrés extremo elevó drásticamente la demanda a 500 req/s empleando un mix de 80% archivos pequeños y 20% archivos grandes, expandiendo defensivamente la infraestructura del servidor a un pool de 12 hilos *worker* y manteniendo el buffer circular con una capacidad estricta de 16 slots. Los indicadores claves se exponen en la Tabla V.

Métrica	Política FIFO (n=50)	Política SFF (n=50)
Latencia media (ms)	0.491 ± 0.002	0.490 ± 0.002
Mediana de latencia (ms)	0.491	0.490
Percentil 95 (p95) (ms)	0.495	0.495
Percentil 99 (p99) (ms)	0.497	0.496
Coefficiente de Variación (CV)	0.00442	0.00452
Throughput (req/s)	499.995 ± 0.003	499.996 ± 0.003

TABLA V. Estadísticos de rendimiento bajo estrés extremo — Escenario C.

La aplicación del test Mann-Whitney U identificó variaciones estadísticamente significativas tanto en latencia ($p = 0.0395$, $r = -0.236$ a favor de FIFO) como en throughput ($p = 0.0132$, $r = +0.286$ a favor de SFF). Al igual que en los escenarios anteriores, el tamaño del efecto pequeño ratifica que estas diferencias carecen por completo de relevancia en entornos reales de producción.

De manera sobresaliente, **se refuta en su totalidad la hipótesis de inanición (*starvation*) o degradación masiva para archivos grandes bajo el scheduler SFF**. El servidor web concurrente demostró una resiliencia sobresaliente al despachar con éxito el 100% de la carga impuesta (30,000 solicitudes por cada réplica de 60 segundos), operando con una tasa de error nula (0%) en las 100 ejecuciones combinadas.

Un hallazgo de alto valor arquitectónico es que la latencia media global en el Escenario C (~ 0.49 ms) resultó ser **notablemente inferior** a la registrada en el Escenario B (~ 0.55 ms), a pesar de estar procesando una tasa de entrada 5 veces superior. Este fenómeno se atribuye directamente a dos variables estructurales:

1. El incremento a 12 hilos *worker* escaló la capacidad de procesamiento paralelo real del threadpool, absorbiendo ráfagas concurrentes de forma eficiente.
2. La reducción del ratio de archivos de 500 KB (de un 30% en B a un 20% en C) disminuyó de forma sustancial la contención y saturación en el subsistema de entrada/salida de disco y red, permitiendo un vaciado acelerado del buffer compartido.

D. Fuentes de error y validez

Para garantizar el rigor metodológico y validar las pruebas inferenciales aplicadas, se ejecutó el test de normalidad de Shapiro-Wilk sobre los conjuntos de datos de latencia. Los resultados arrojaron valores de $p < 0.05$ en la gran mayoría de las agrupaciones (a excepción del grupo FIFO en el Escenario C, el cual exhibió un comportamiento normal con $p = 0.1760$), confirmando de manera sólida que las latencias web violan los supuestos gaussianos clásicos y justificando plenamente el uso exclusivo de estadística no paramétrica.

Durante la fase de auditoría e inspección profunda de datos, se aislaron anomalías severas en la métrica de equidad interna (*fairness_cv*), que mide la variabilidad intra-réplica de solicitudes individuales. Específicamente, se identificaron picos atípicos extremos en tres ejecuciones del scheduler SFF: la réplica 4 del Escenario B ($CV = 4.584$), junto a las réplicas 6 ($CV = 2.374$) y 28 del Escenario C ($CV = 2.471$). Mientras que el comportamiento estándar se

mantuvo óptimo y estable por debajo de 0.4, estas réplicas anómalas representan órdenes de magnitud superiores.

Dado que las condiciones de carga de la suite Python se mantuvieron estables mediante el mecanismo de *token bucket*, esta variabilidad exógena se atribuye de forma directa a dinámicas incontrolables del sistema operativo anfitrión. Entre ellas destacan las interrupciones imprevistas del kernel de Linux, fluctuaciones inevitables (*jitter*) en la interfaz de red *loopback* y la planificación del scheduler CFS de Linux sobre los núcleos de ejecución asignados. Por último, se reconoce el impacto del *Global Interpreter Lock* (GIL) de CPython y los límites en descriptores de archivos concurrentes como restricciones físicas de la infraestructura de pruebas para escalar de manera limpia más allá del umbral de las 500 req/s.

VII. CONCLUSIONES

Este proyecto demostró que la transformación de un servidor web monohilo a un modelo productor-consumidor multi-hilo con POSIX pthreads es técnicamente viable y produce mejoras sustanciales en throughput bajo carga concurrente real. La implementación de dos políticas de scheduling en el mismo buffer compartido evidenció el tradeoff fundamental entre eficiencia media (SFF) y equidad/ausencia de inanición (FIFO).

La colisión de nombres entre el enum de scheduling y las macros POSIX (`SCHED_FIFO` de `<sched.h>`) constituyó un recordatorio práctico de las complejidades del espacio de nombres en C de bajo nivel: en programación de sistemas, la elección de identificadores debe anticipar conflictos con macros del sistema operativo incluidas transitivamente.

La separación arquitectónica entre `request_parse()` (acceptor) y `request_serve()` (worker) resultó ser el diseño más limpio para habilitar concurrencia sin condiciones de carrera sobre el parseo HTTP: el acceptor es el único productor del buffer, y el estado necesario para servir la petición viaja completamente encapsulado en `request_t`.

Los resultados experimentales confirmaron que la política SFF no produjo mejoras significativas en la latencia media global respecto a FIFO, demostrando que en el sistema implementado el rendimiento está dominado por las operaciones de E/S de bajo nivel y no por el algoritmo de scheduling. Asimismo, se refutó la

hipótesis de inanición para archivos grandes en el escenario de saturación, evidenciando una robustez operativa inesperada ante cargas altas.

REFERENCIAS

- [1] R. H. Arpaci-Dusseau y A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, v1.10. Arpaci-Dusseau Books, 2023. [En línea]: <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- [2] R. H. Arpaci-Dusseau, "ostep-projects: Projects for an Operating Systems class." GitHub. [En línea]: <https://github.com/remzi-arpacidusseau/ostep-projects>
- [3] The Open Group, "pthread.h — threads", POSIX.1-2017 (IEEE Std 1003.1-2017). [En línea]: <https://pubs.opengroup.org/onlinepubs/9699919799/>
- [4] W. R. Stevens y S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley, 2013.
- [5] B. W. Kernighan y D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.
- [6] E. W. Dijkstra, "Cooperating Sequential Processes," en *Programming Languages*, F. Genuys, Ed. Academic Press, 1968, pp. 43-112.